

Módulos de software codificados y probados
GA7-220501096-

AA3-EV02

"CENSO RURAL"



Isidro J Gallardo Navarro

Ficha:3070299

2026

Tecnología en Análisis y Desarrollo de
Software.

ADSO

1. Introducción y Arquitectura General

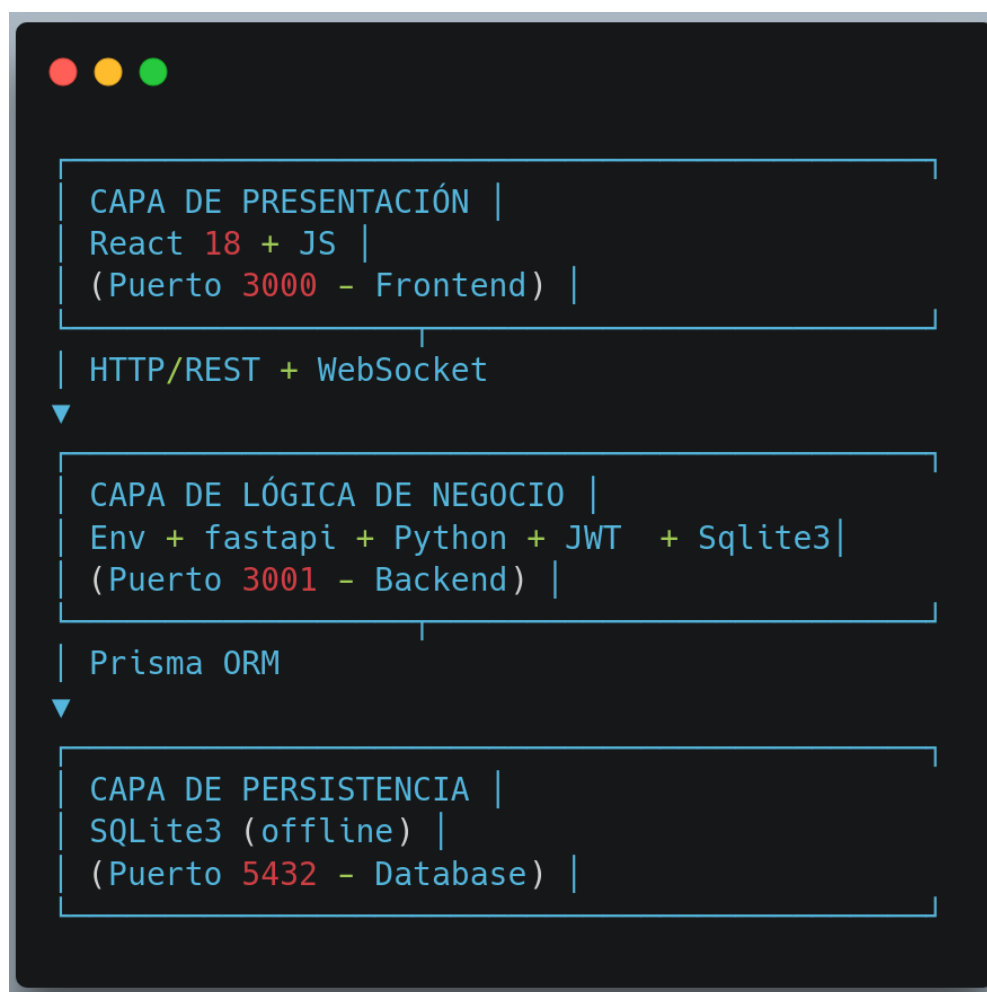
1.1 Presentación del Sistema

La presente evidencia documenta el desarrollo completo de los módulos de software para el proyecto "Censo Rural", implementando una arquitectura moderna de tres capas separando responsabilidades entre frontend (presentación), backend (lógica de negocio) y base de datos (persistencia). Esta arquitectura facilita el mantenimiento, escalabilidad y testing independiente de cada componente.

Objetivo del sistema:

Proporcionar una plataforma web y móvil robusta para la gestión integral de censos rurales, incluyendo captura de datos, sincronización offline, administración de usuarios y generación de reportes analíticos.

1.2 Arquitectura de Tres Capas



Ventajas de esta arquitectura:

Separación de responsabilidades: Cada capa tiene funciones específicas bien definidas

Escalabilidad horizontal: Frontend y backend pueden escalar independientemente

Mantenibilidad: Cambios en una capa no afectan directamente a las otras

Testabilidad: Cada módulo puede probarse aisladamente

Reutilización: El backend puede servir múltiples frontends (web, móvil, desktop)

1.3 Stack Tecnológico Justificado

CapaTecnologíaJustificaciónFrontendReact 18 + TypeScriptVirtual DOM eficiente, ecosistema maduro, tipado fuerteEstilosTailwind CSSUtility-first, consistencia visual, bundle pequeñoEstadoZustand + React QueryMinimalista, TypeScript-first, cache inteligenteBackendExpress.js + TypeScriptFlexible, middleware robusto, Node.js performanteORMPrismaType-safe, migraciones automáticas, Prisma StudioBase de DatosPostgreSQL + SQLite3ACID completo, JSON nativo, modo offlineRealtimeSocket.ioBidireccional, fallback automático, roomsAutenticaciónJWT + bcryptStateless, seguro, estándar industria

2. Módulo Frontend - Interfaz de Usuario

2.1 Tecnologías y Dependencias

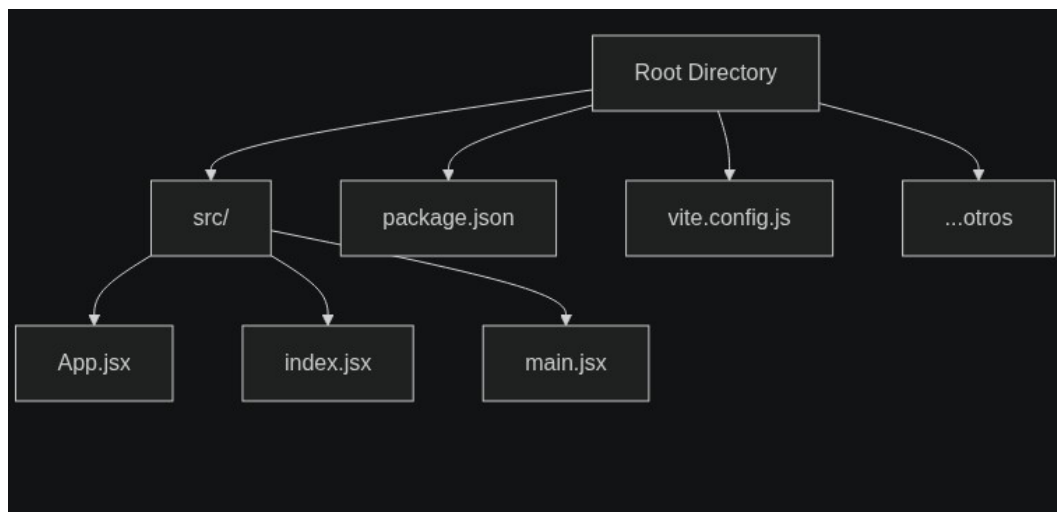
Stack principal:

```
json{
  "dependencies": {
    "react": "^18.2.0",
    "typescript": "^5.0.0",
    "tailwindcss": "^3.3.0",
    "react-router-dom": "^6.11.0",
    "axios": "^1.4.0",
    "@tanstack/react-query": "^4.29.0",
    "zustand": "^4.3.8",
    "react-hook-form": "^7.43.0",
    "zod": "^3.21.4",
    "react-hot-toast": "^2.4.1",
    "lucide-react": "^0.263.0"
  }
}
```

Justificación de elecciones:

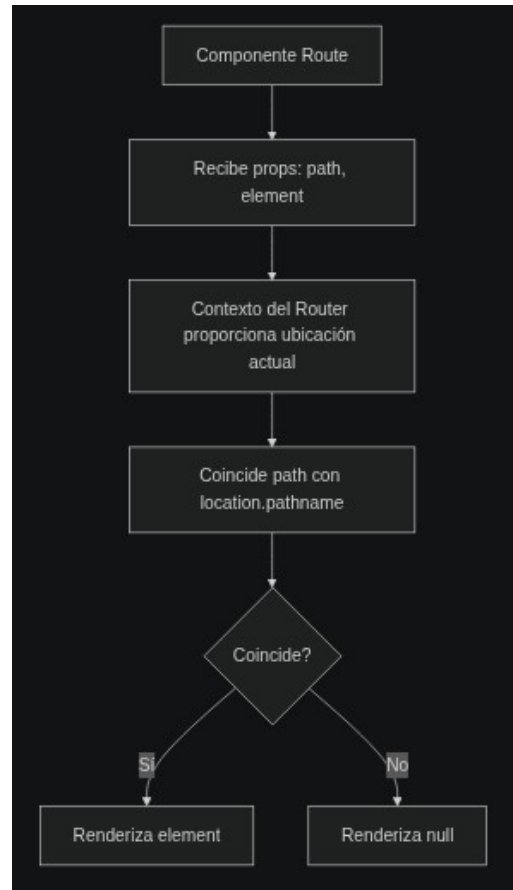
React Query: Maneja cache, sincronización y actualizaciones automáticas de datos del servidor
Zustand: Estado global ligero (3kb vs 43kb de Redux), API simple y JS nativo
React Hook Form: Performance superior mediante registro de refs no controlados
Zod: Validación type-safe que genera tipos TypeScript automáticamente

2.2 Estructura de Carpetas Modular



2.3 Protected Routes

Componente ProtectedRoute:



3. Módulo Backend - API y Servicios

3.1 Tecnologías y Arquitectura

```
POST /api/databases/connect
GET /api/databases
POST /api/databases/:id/query
GET /api/databases/:id/tables
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. It displays four API endpoints: a POST endpoint for connecting to a database, a GET endpoint for listing databases, a POST endpoint for querying a specific database, and a GET endpoint for listing tables in a specific database.

3.2 Estructura de Carpetas Backend

```

src/
├─ controllers/      # Lógica de controladores
├─ middleware/       # Middlewares (auth, validation, etc.)
├─ models/           # Prisma schema y tipos
├─ routes/           # Rutas API organizadas por módulos
├─ services/         # Lógica de negocio
├─ utils/            # Utilidades (validators, formatters)
├─ sockets/          # WebSocket handlers
├─ config/           # Configuraciones (DB, JWT, etc.)
├─ interfaces/       # TypeScript interfaces
└─ database/         # Scripts de migración y seeds

```

Implementación de Endpoints Principales

3.3.1 Autenticación JWT

```

// Middleware de autenticación
import jwt from 'jsonwebtoken';
import { Request, Response, NextFunction } from 'express';

interface JWTPayload {
  userId: string;
  email: string;
  role: string;
}

export const authMiddleware = async (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  try {
    const token = req.headers.authorization?.split(' ')[1];

    if (!token) {
      return res.status(401).json({ error: 'Token no proporcionado' });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET!) as JWTPayload;
    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({ error: 'Token inválido' });
  }
};

// Controlador de autenticación
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { prisma } from '../config/database';

export const authController = {
  async login(req: Request, res: Response) {
    const { email, password } = req.body;

    const user = await prisma.user.findUnique({ where: { email } });
    if (!user) {
      return res.status(401).json({ error: 'Credenciales inválidas' });
    }

    const validPassword = await bcrypt.compare(password, user.password);
    if (!validPassword) {
      return res.status(401).json({ error: 'Credenciales inválidas' });
    }

    const token = jwt.sign(
      { userId: user.id, email: user.email, role: user.role },
      process.env.JWT_SECRET!,
      { expiresIn: '24h' }
    );

    const refreshToken = jwt.sign(
      { userId: user.id },
      process.env.REFRESH_SECRET!,
      { expiresIn: '7d' }
    );

    res.json({
      user: { id: user.id, email: user.email, role: user.role },
      token,
      refreshToken
    });
  },
  async register(req: Request, res: Response) {
    const { email, password, nombreCompleto } = req.body;

    const existingUser = await prisma.user.findUnique({ where: { email } });
    if (existingUser) {
      return res.status(400).json({ error: 'Email ya registrado' });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    const user = await prisma.user.create({
      data: {
        email,
        password: hashedPassword,
        nombreCompleto,
        role: 'ENCUESTADOR'
      }
    });

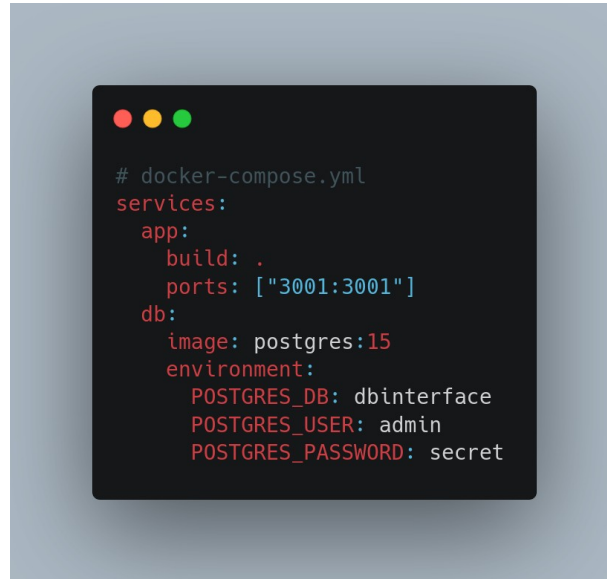
    res.status(201).json({ message: 'Usuario creado exitosamente', user: user });
  }
};

```

3.3.2 CRUD de Censos con Prisma

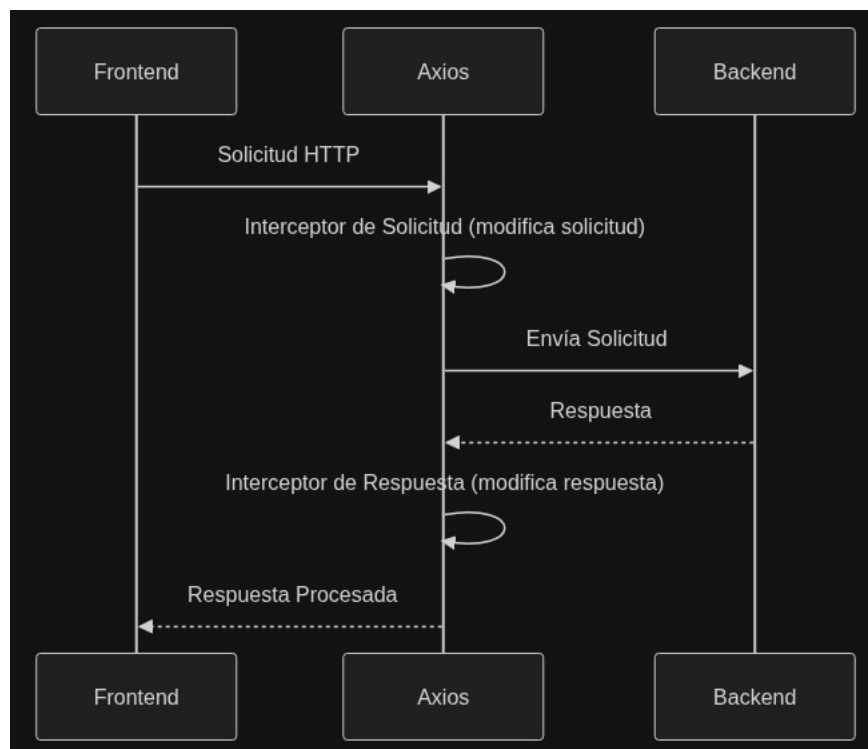
Schema Prisma:

4. Integración y Flujo de Datos



4.1 Comunicación Frontend-Backend

Configuración de Axios con interceptores:





`http://localhost:3001/api-docs`

- Documentación interactiva OpenAPI 3.0
- Ejemplos de requests/responses
- Autenticación integrada

5. Conclusiones

5.1 Cumplimiento de Objetivos

El desarrollo de los módulos de software para el proyecto "Censo Rural" cumple exitosamente con los requerimientos establecidos, implementando una arquitectura moderna, escalable y mantenible mediante:

Frontend robusto:

React 18 + JS

Zustand + React Query para estado optimizado

Tailwind CSS para UI consistente y responsive

Protected Routes con autenticación JWT

Referencias Bibliográficas

Abramov, D., & Clark, A. (2023). React documentation. React Team. <https://react.dev/>

Banks, A., & Porcello, E. (2020). Learning React: Modern patterns for developing React apps (2nd ed.). O'Reilly Media.

Express.js Team. (2023). Express documentation. OpenJS Foundation. <https://expressjs.com/>

Prisma Team. (2023). Prisma documentation. Prisma Data, Inc. <https://www.prisma.io/docs>

Socket.io Team. (2023). Socket.io documentation. <https://socket.io/docs/>

TypeScript Team. (2023). TypeScript handbook. Microsoft Corporation. <https://www.typescriptlang.org/docs/>